I/O fundamentals

Introduction

- The I/O subsystem provides an efficient mode of communication between CPU and outside environment
- Devices that are under the direct control of computer are said to be connected on-line
- Input or output devices attached to the computer are also referred as peripherals.
- I/O interface or I/O Module provides a method for transferring information between internal storage and external i/o devices.

PERIPHERAL DEVICES

Input Devices

- Keyboard
- Optical input devices
  - Card Reader
  - Paper Tape Reader
  - Bar code reader
  - Digitizer
  - Optical Mark Reader
- Magnetic Input Devices
  - Magnetic Stripe Reader
- Screen Input Devices
  - Touch Screen
  - Light Pen
  - Mouse
- Analog Input Devices

Output Devices

- Card Puncher, Paper Tape Puncher
- CRT
- Printer (Impact, Ink Jet, Laser, Dot Matrix)
- Plotter
- Analog
- Voice

I/O interface

Resolves the *differences* between the computer and peripheral devices

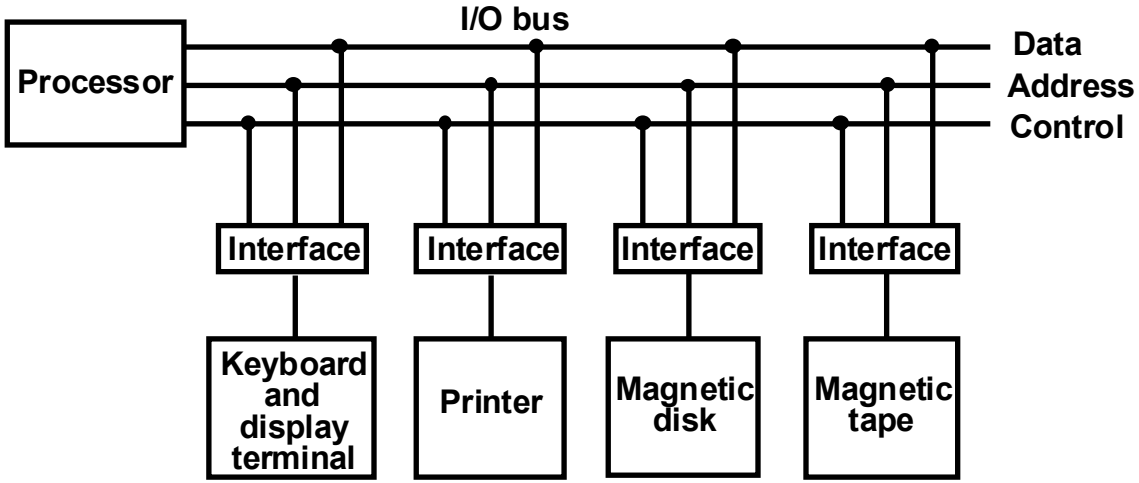
- **Peripherals - Electromechanical Devices**
- **CPU or Memory - Electronic Device**

- **- Data Transfer Rate**
- **Peripherals - Usually slower**
- **CPU or Memory - Usually faster than peripherals**
- **Some kinds of Synchronization mechanism may be needed**

- **- Unit of Information**
- **Peripherals - Byte**
- **CPU or Memory - Word**

- **- Operating Modes**
- **Peripherals - Autonomous, Asynchronous**
- **CPU or Memory - Synchronous**

I/O BUS AND INTERFACE MODULES



Each peripheral has an interface module associated with it

Interface

- Decodes the device address (device code)
- Decodes the commands (operation)
- Provides signals for the peripheral controller
- Synchronizes the data flow and supervises the transfer rate between peripheral and CPU or Memory

Typical I/O instruction

Op. code	Device address	Function code
		(Command)

Buffering

I/O buffering is a technique used in computers to manage data transfer between the computer's memory and input/output devices (like hard drives, printers, or network devices).

It helps make data transfer more efficient by temporarily storing data in a buffer, which is a reserved area of memory.

It involves the temporary storage of data in a buffer, which is a reserved area of memory, to reduce the number of I/O operations and manage the flow of data between fast and slow devices or processes

Types of IO Buffer

1. Single Buffer
2. Double Buffer
3. Circular Buffer

I/O Bus and Memory Bus

Functions of Buses

MEMORY BUS **is for information transfers between CPU and the Main Memory**

* ***I/O BUS*** **is for information transfers between CPU and I/O devices through their I/O interface**

* **Many computers use a common single bus system for both memory and I/O interface units**
  - **Use one common bus but separate control lines for each function**
  - **Use one common bus with common control lines for both functions**

* **Some computer systems use two separate buses,**
  one to communicate with memory and the other with I/O interfaces

Isolated I/O

Isolated I/O

- Separate I/O read/write control lines in addition to memory read/write control lines.
- Separate (isolated) memory and I/O address spaces .
- Distinct input and output instructions.
- When CPU fetches and decodes the opcode of an I/O instruction, it places the address into the common address lines. Also enables read/write control lines => the address in the address lines is for interface register and not for a memory word.

Memory Mapped I/O

Memory mapped I/O

- A single set of read/write control lines

  (no distinction between memory and I/O transfer)

- Memory and I/O addresses share the common address space

  -> reduces memory address range available

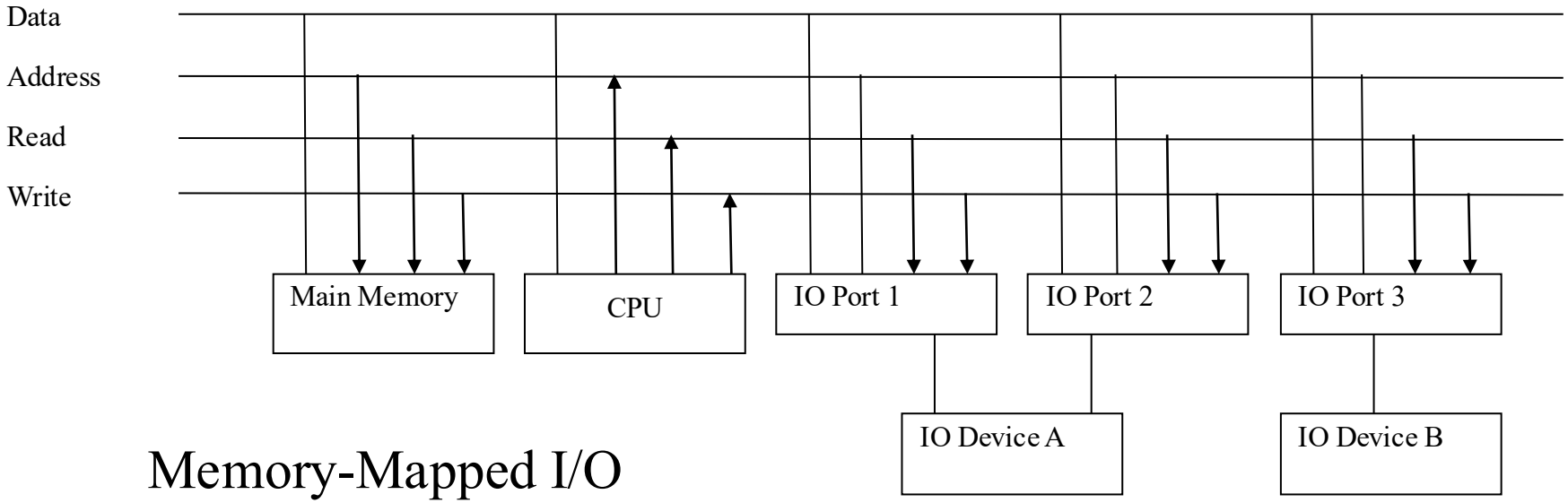
  -> Memory addresses xffff0000 and above are used for I/O devices

- No specific input or output instruction

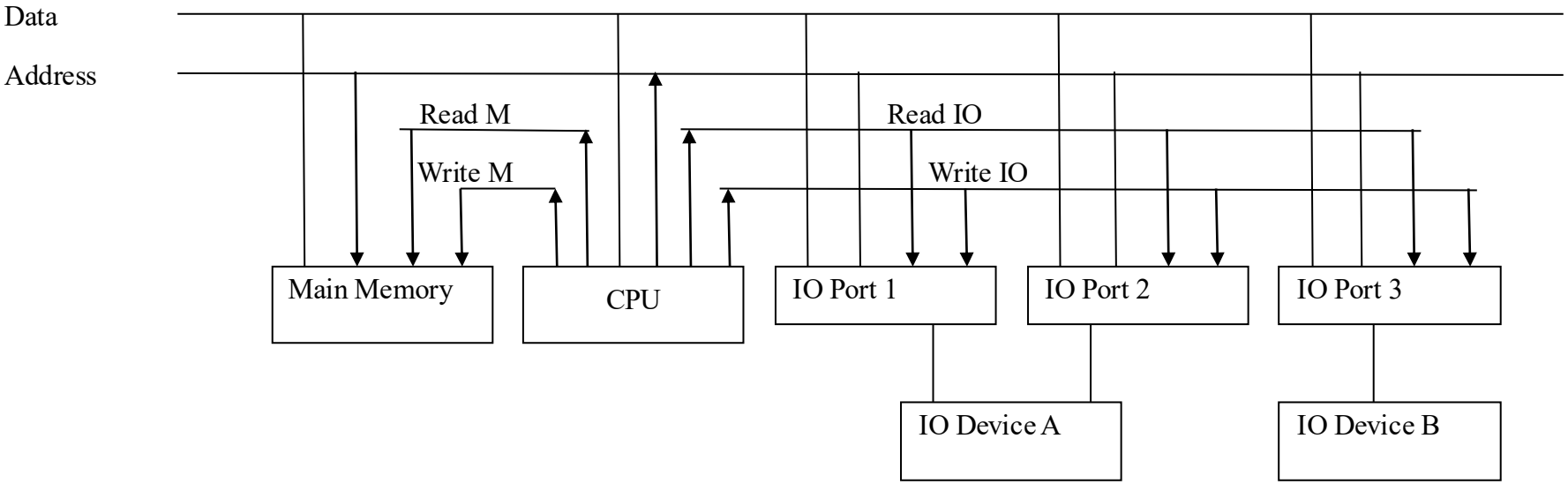
  -> The same memory reference instructions can be used for I/O transfers

  -> when the bus sees certain addresses, it knows they are not memory addresses, but are addresses for accessing I/O devices.

- Considerable flexibility in handling I/O operations



Memory-Mapped I/O



I/O Mapped I/O

Asynchronous Data Transfer

Synchronous and asynchronous operations

Synchronous - All devices derive the timing information from common clock line

Asynchronous - No common clock

Asynchronous Data Transfer

Asynchronous data transfer between two independent units requires that *control signals* be transmitted between the communicating units *to indicate the time at which data is being transmitted*

Two Asynchronous Data Transfer Methods

Strobe pulse

- A strobe pulse is supplied by one unit to indicate the other unit when the transfer has to occur

Handshaking

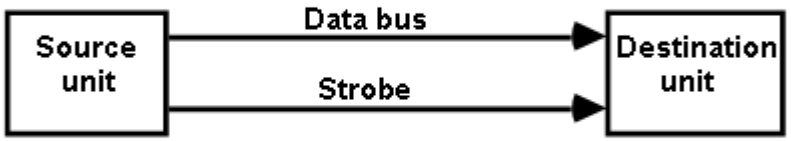
- A control signal is accompanied with each data being transmitted to indicate the presence of data
- The receiving unit responds with another control signal to acknowledge receipt of the data

Strobe Control

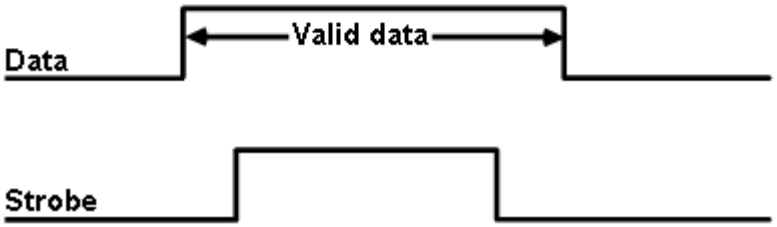
- **Employs a single control line to time each transfer**
- **The strobe may be activated by either the source or the destination unit**
- **Data bus carries the binary information from source to destination**

Source-Initiated Strobe for Data Transfer

Block Diagram

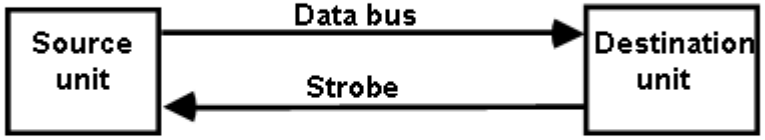


Timing Diagram

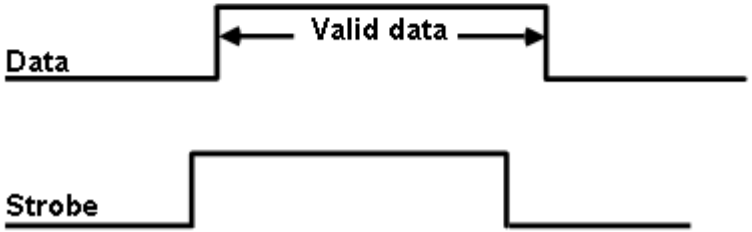


Destination-Initiated Strobe for Data Transfer

Block Diagram



Timing Diagram



Source initiated strobe

- It could be a memory write control signal from the CPU to a memory unit.
- Source is the CPU, places a word on the data bus and informs the memory unit which is destination, that this is a write operation
- Disadvantage – no way of knowing whether the destination unit has actually received data

Destination initiated strobe

- It could be a memory read control signal.
- CPU, the destination initiates the read operation to inform the memory which is the source to place a selected word into the data bus.
- Disadvantage - no way of knowing whether the source has actually placed the data on the bus

Hand Shaking

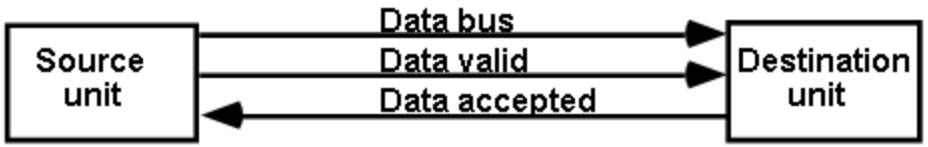
- To solve the disadvantages of strobe, the *HANDSHAKE* method introduces a second control signal to provide a *Reply* to the unit that initiates the transfer.

- 2 types
  - Source initiated transfer
  - Destination initiated transfer

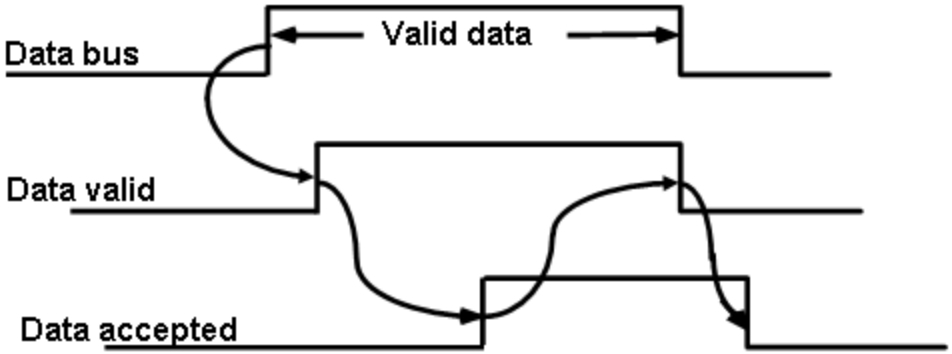
- Provides ***high degree of flexibility*** **and** ***reliability***
- Incompletion of data transfer can be detected by means of a ***timeout mechanism***
- The timeout signal can be used to interrupt the processor and hence execute a service routine that takes appropriate error recovery action.

SOURCE-INITIATED TRANSFER USING HANDSHAKE

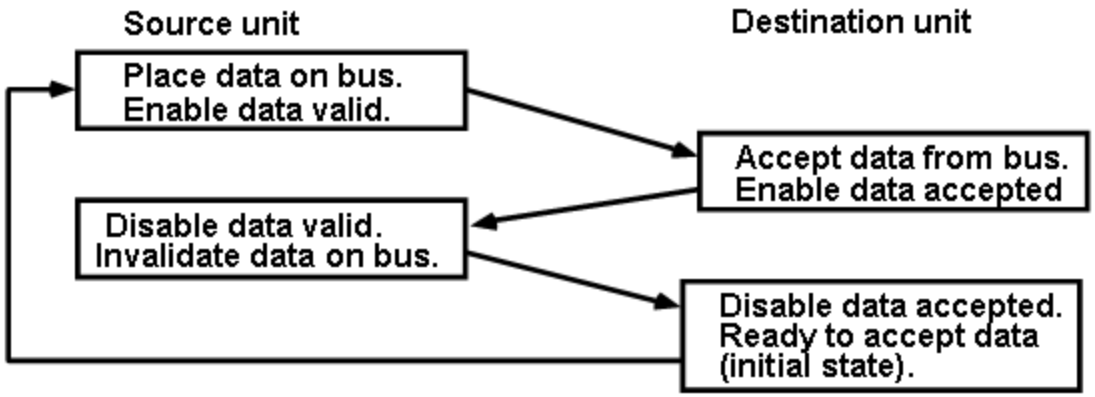
Block Diagram



Timing Diagram

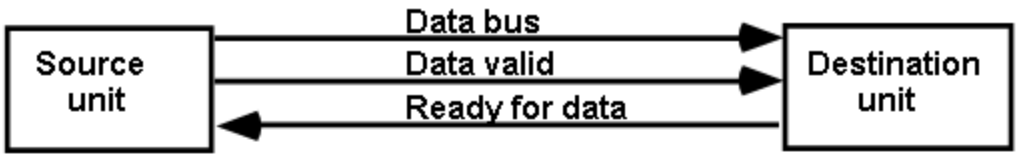


Sequence of Events

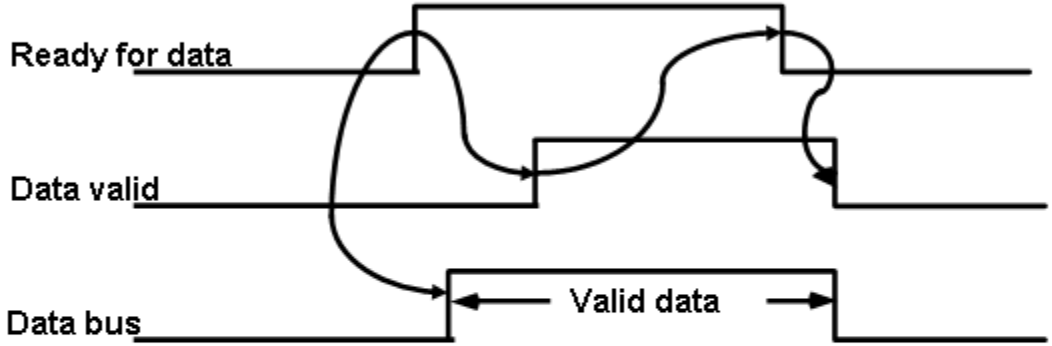


DESTINATION-INITIATED TRANSFER USING HANDSHAKE

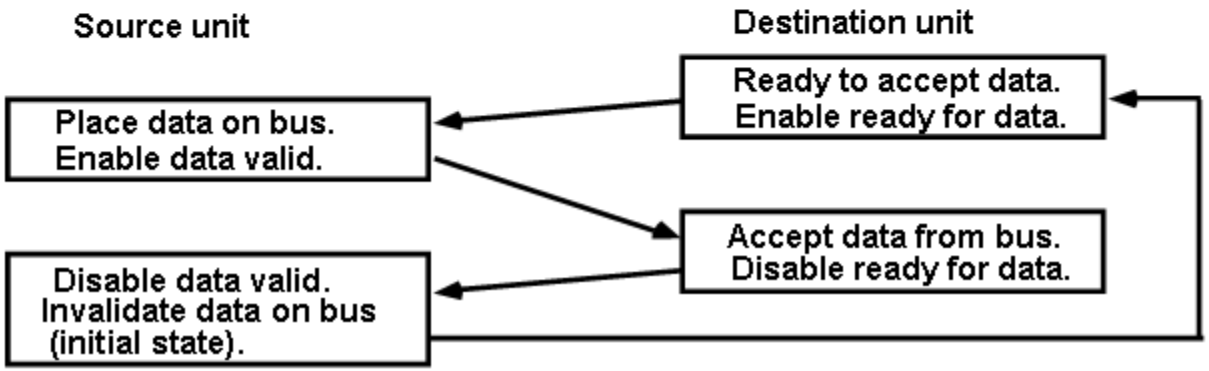
Block Diagram



Timing Diagram



Sequence of Events



Modes of Transfer

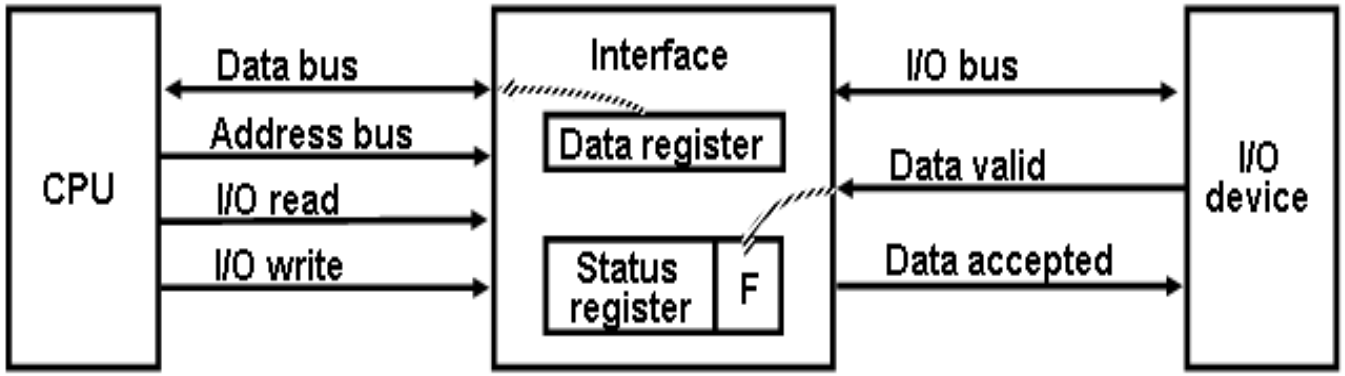
- 3 different **Data Transfer Modes** between the central computer (CPU or Memory) and peripherals;
  - Program-Controlled I/O or programmed I/O
  - Interrupt-Initiated I/O or Interrupt driven I/O
  - Direct Memory Access (DMA)
- **Programmed I/O**
  - Transferring data under program control requires constant monitoring of the peripheral by CPU.
  - I/O device does not have direct access to memory
  - Programmed I/O is time consuming process since it keeps the processor busy needlessly.

Programmed I/O

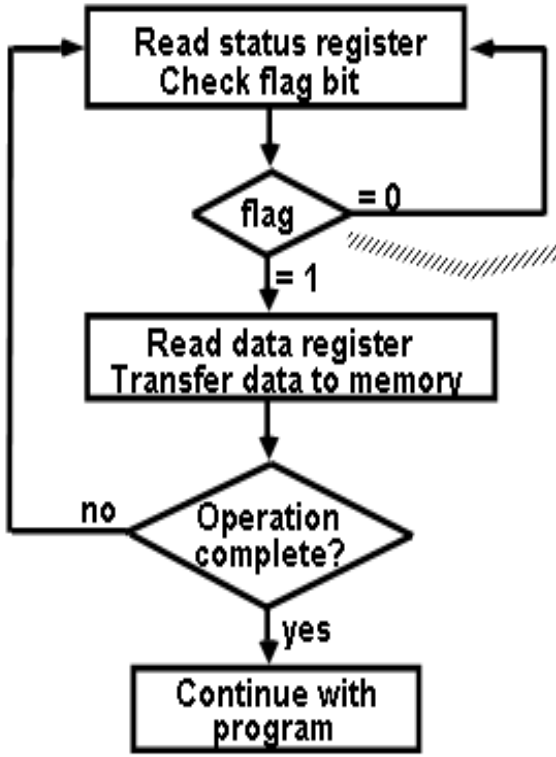
- More instructions may be required during the transfer of data from I/O device to memory
  - Input instruction: from I/P device to CPU
  - Store instruction: CPU->memory
  - To verify the data availability
  - To count the number of words transferred
- A common programming task is to transfer a block of words from an I/O device and store them in a memory buffer
- Useful in small low speed systems or in dedicated systems to monitor a device continuously.

Programmed I/O

Data Transfer from I/O device to CPU



Flow chart for CPU program to input data



Polling or Status Checking

- Continuous CPU involvement
- CPU slowed down to I/O speed
- Simple
- Least hardware

Interrupt

- An interrupt is a signal from a device attached to a computer or from a program within the computer that causes the CPU to stop its normal program execution and perform service related to the event.
- Examples of interrupts :I/O completion, divide-by-0, etc.
- **Maskable Interrupt**: It is a hardware interrupt that may be ignored by setting a bit in an interrupt mask register's (IMR) bit-mask.
- **Non-maskable Interrupt**: is a hardware interrupt that does not have a bit-mask associated with it - meaning that it can never be ignored. NMIs are often used for timers, especially watchdog timers

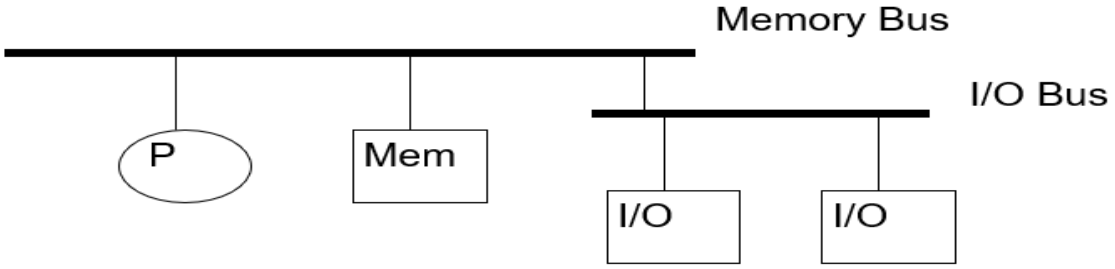
Interrupt-Initiated I/O

- When the interface determines that the I/O device is ready for data transfer, it generates an *Interrupt Request* to the CPU .

- Upon detecting an interrupt, CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing

Interrupt-Driven I/O (Overview)

- CPU setups and start I/O operation
- CPU goes off to do other things
- When I/O is done, CPU is interrupted
- CPU handles the interrupt
- CPU resumes interrupted operation
- => Interrupts save overhead of polling.



Handling Interrupts

- Device raises the interrupt (sends interrupt signal)
- At the end of the current fetch execute cycle, CPU checks for interrupt, if there is one, it saves its status (register values)
- CPU identifies interrupting device
- Given the interrupting device number, use this to map into the Interrupt vector table
  - For instance, interrupt 15 would be at location 15 in memory, this entry stores the location of interrupt 15's handler in operating system memory
- Move this address into the PC
- Execute interrupt handler
- Upon completion, restore saved register values

- 2 methods in which the CPU chooses the branch address of the service routine
  - Non-vectored interrupt
    - Branch address is assigned to a fixed location
    - User has to provide address of subroutine using a CALL instruction
  - Vectored interrupt
    - Interrupt supplies the branch info (direct or indirect address of ISR) to the CPU
    - Processor automatically generates the subroutine address

Priority Interrupt

- Establishes priority over the various sources to determine which condition is to be serviced first when two or more requests arrive simultaneously.
- The system may also determine which conditions are permitted to interrupt the computer while another interrupt is being serviced.
- Priority can be established by software or hardware

Interrupt Cycle

The Interrupt enable flip-flop (IEN) can be set or cleared by program instructions.

A programmer can therefore allow interrupts (clear IEN) or disallow interrupts (set IEN)

At the end of each instruction cycle the CPU checks IEN and IST. If either is equal to zero, control continues with the next instruction. If both = 1, the interrupt is handled.

Interrupt micro-operations:

SP←SP – 1	(Decrement stack pointer)
M[SP] ← PC	Push PC onto stack
INTACK ← 1	Enable interrupt acknowledge
PC ←VAD	Transfer vector address to PC
IEN ←0	Disable further interrupts

Go to fetch next instruction

Interrupt Service Routine

An interrupt handler, also known as an interrupt service routine (ISR), is a callback subroutine in an operating system whose execution is triggered by the reception of an interrupt. Interrupt handlers have a multitude of functions, which vary based on the reason the interrupt was generated.

Interrupt Overhead

The interrupt overhead is caused by context switching (storing and restoring the state of CPU)

On interrupt handler entry, the context of the current process and its thread must be saved. On exit, it must be restored.

On handler entry, memory locations different from the memory locations in the cache are used, and therefore cache updates are required.

Role of CPU in transfer of information

peripheral device -> CPU -> memory

⇒CPU limits the speed of transfer

Peripheral device -> memory

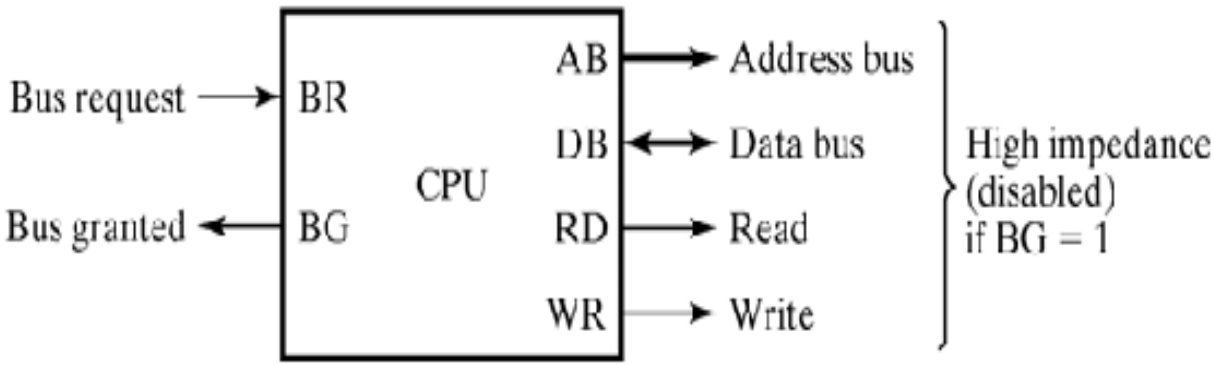
⇒Transfer speed increases

⇒Peripheral device manage the memory bus directly.

⇒DMA (Direct memory Access)

⇒CPU is idle

CPU Bus Signals for DMA Transfer



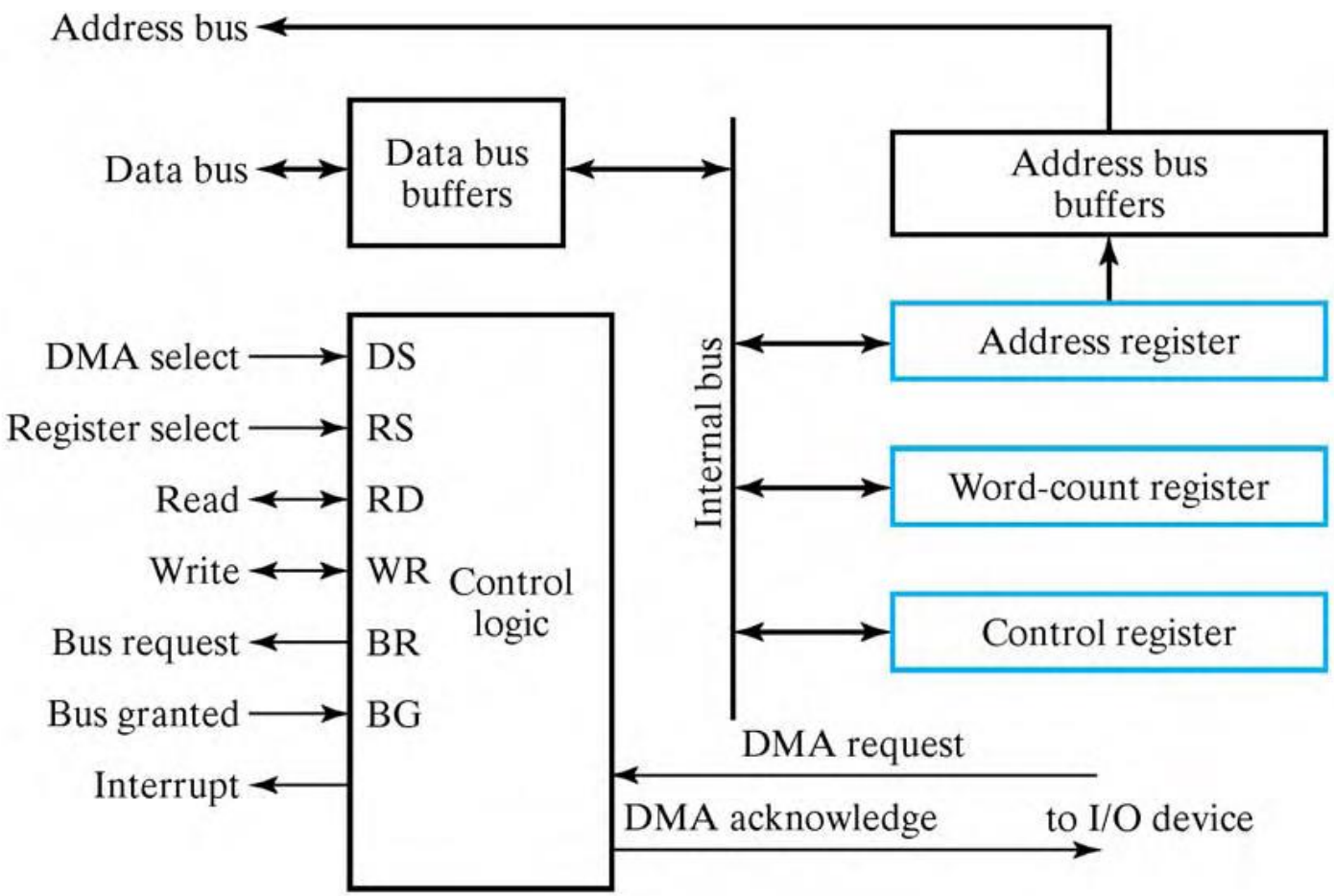
- DMA Controller sends Bus Request to CPU
- CPU stops execution of current instruction and places address bus, data bus, read and write lines into high impedance state.
- CPU activates Bus Grant
- After transfer DMA disables BR
- CPU continues normal operation

Different ways of DMA transfer

- Burst transfer – a block sequence consisting of a number of memory words is transferred in a continuous burst => need for fast devices
- Transfer Rate=Block Size/ Block Cycle time

- Cycle stealing – transfer one word at a time, after which it must return control of the buses to the CPU.
  - CPU delays 1 cycle to allow Direct Memory I/O transfer
- Transfer Rate=Word Size/ Bus Cycle time

DMA Controller



DMA Controller contd.,

- Three registers
  - Address registers – to specify the desired location in memory
    - Incremented after each word transfer
  - Word count registers – numbers of words to be transferred
    - Decremented after each word transfer and tested for 0.
  - Control register – specifies mode of transfer

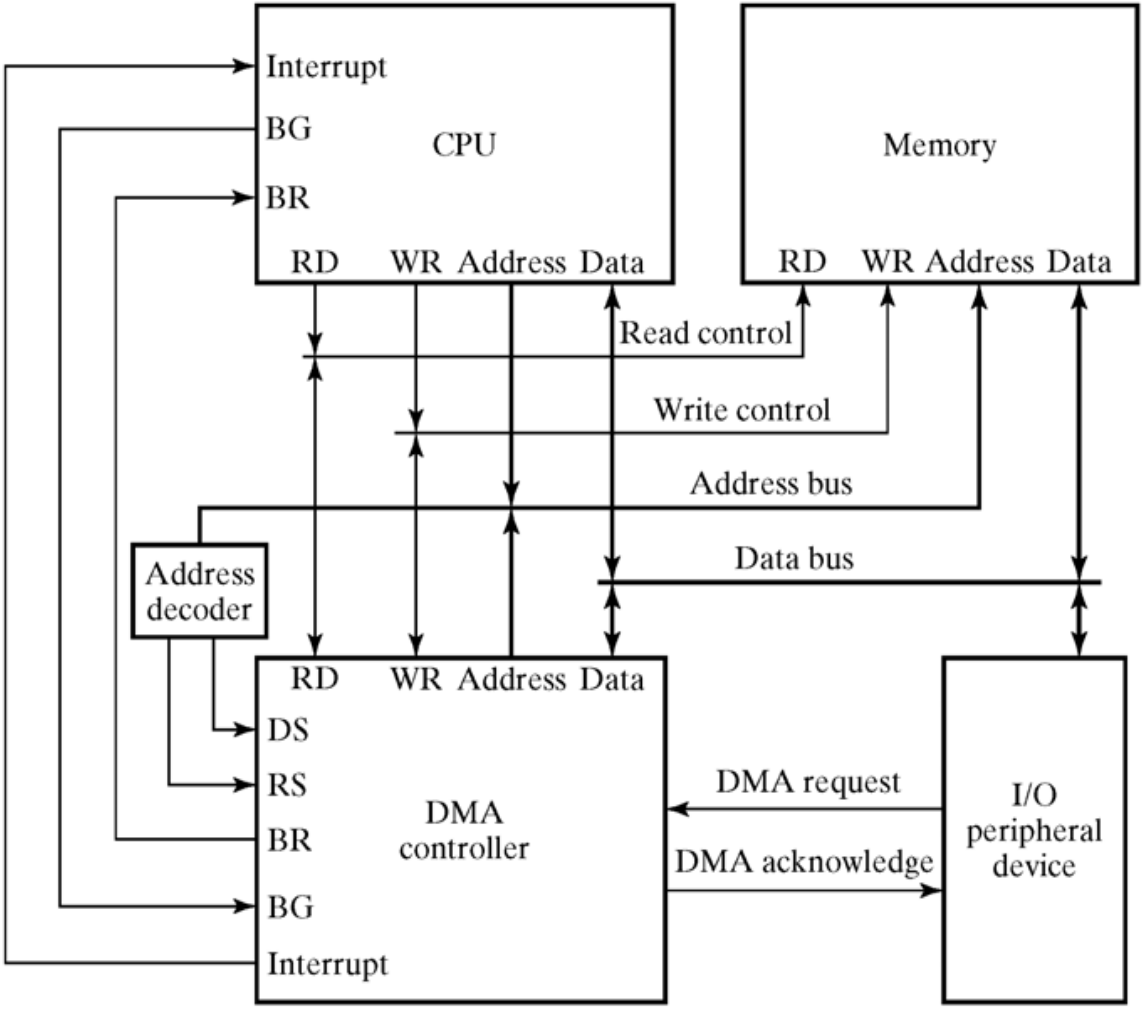
DMA Controller

- Data bus and control lines => used to communicate with CPU
- Registers in DMA are selected by CPU through address bus by enabling DS and RS inputs.
- BG = 0 => CPU reads from or writes to the DMA registers
- BG = 1 => DMA directly communicates with Memory by specifying address in address bus and activates the RD or WR control.
- Request and acknowledge signals are used to communicate with peripheral devices

Initialization of DMA

- CPU initializes DMA by sending the following information through data bus
  - Starting address of the memory block where data are available (for read) or where data are to be stored (for write)
  - The word count
  - Control – to read or write
  - A control to start the DMA transfer
- After initialization, CPU stops communicating with DMA unless it receives an interrupt signal or if it wants to checks how many words have been transferred.

DMA Transfer



DMA Transfer

- Peripheral device ---DMA Request---> DMA controller ---Bus Request---> CPU ---Bus grant---> DMA controller
- DMA controller puts the current value of its address register onto address bus, activates RD or WR signal, and ---DMA Acknowledge---> peripheral device
- RD and WR are bidirectional
  - BG = 0 => CPU communicates with the internal DMA registers
  - BG = 1 => RD and WR are output lines from DMA controller to the RAM to specify read or write operation for data

DMA contd.,

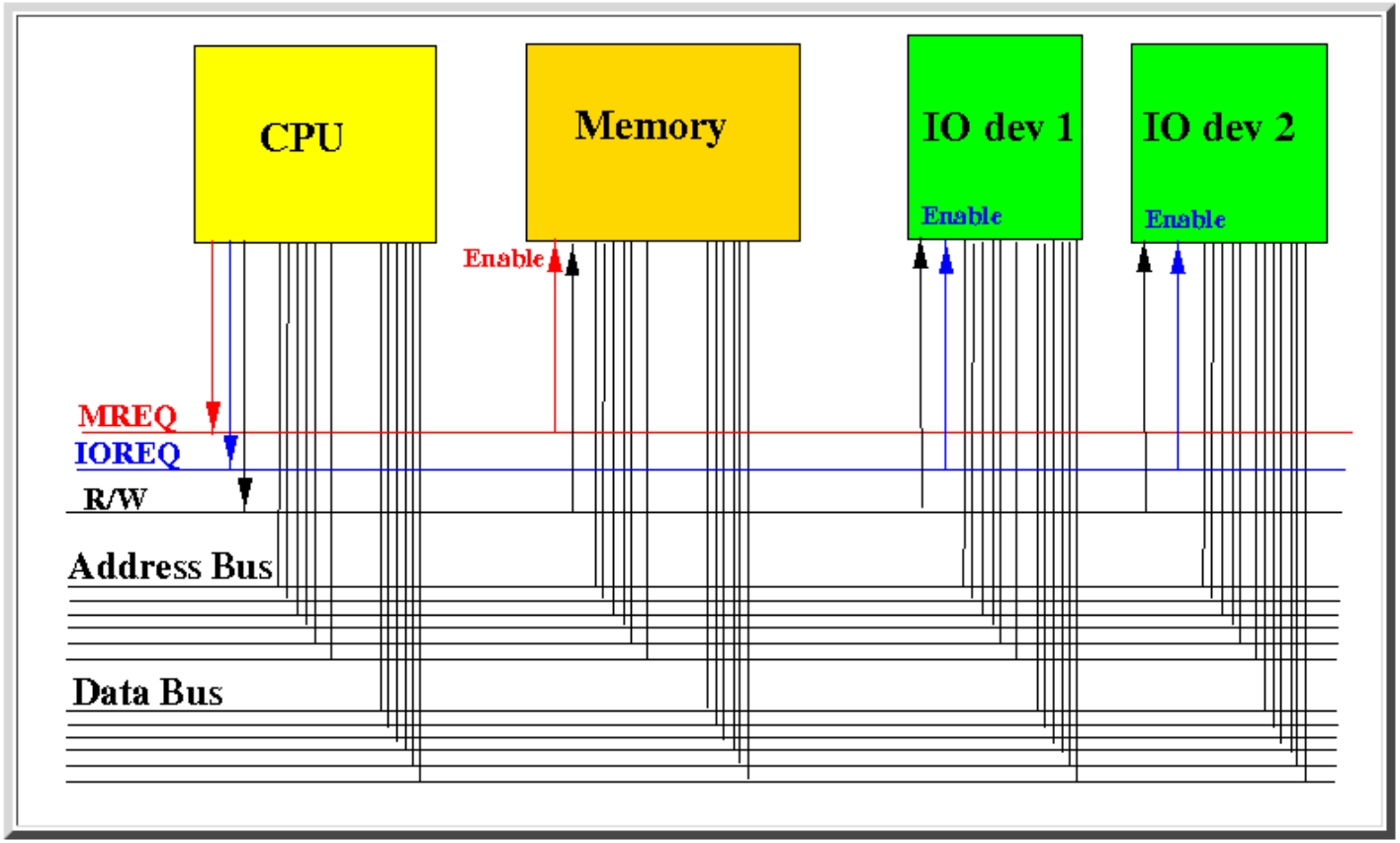
- When the peripheral device receives a DMA acknowledge, it puts a word onto data bus or receives a word from data bus
- For each word transfer, DMA increments address register and decrements word count register.
- If WC = 0, DMA disables Bus request.
- Termination is informed to CPU by means of interrupt signal.
- When CPU receives interrupt signal, it checks value of word count register.

Application

- Fast transfer of information between magnetic disk and memory
- Updating the display in an interactive terminal

Bus System

- How devices are connected inside a compute:



Synchronous Bus

- Synchronous bus (e.g., processor-memory buses)
  - Includes a clock in the control lines and has a fixed protocol for communication that is relative to the clock
  - Advantage: involves very little logic and can run very fast
  - Disadvantages:
    - Every device communicating on the bus must use same clock rate

Asynchronous Bus

- Asynchronous bus (e.g., I/O buses)
  - It is not clocked, so requires a handshaking protocol and additional control lines (ReadReq, Ack, DataRdy)
  - Advantages:
    - Can accommodate a wide range of devices and device speeds
  - Disadvantage: slow(er)

DMA - Problems

1 MB	= 1024 KB	= 2^{20} B
1 GB	= 1024 MB	= 2^{30} B

Problem 1:

A hard disk with a transfer rate of 20 Mbytes/second is constantly transferring data to memory using DMA. The processor runs at 300MHz, and takes 300 and 900 clock cycles to initiate and complete DMA transfer respectively. If the size of the transfer is 20 Kbytes, what is the percentage of processor time consumed for the transfer operation?

Solution:

$$Total\ Time = \frac{Total\ Data}{Total\ Transfer\ Rate}$$

$$Processor\ Time = \frac{Cycles\ for\ DMA\ transfer}{Processor\ Speed}$$

$$\%\ Processor\ Time = \frac{Processor\ Time}{Total\ Time} \times 100$$

Step 1: Find Total Transfer Time

Transfer rate=20 MB per second = 20×2^{20} bytes/sec

Data=20 KB=20×2^{10} bytes

Time=$(20 \times 2^{10})/(20 \times 2^{20}) = 1 \times 2^{-10} = 1 \times 10^{-3}$ =1 ms

Step 2 :Find Processor Time

1 MHz = 10^6 cycles/sec

Processor speed= 300 MHz = 300×10^6 Cycles/sec

Cycles required by CPU for DMA Transfer =300+900 =1200 cycles

Time=1200 cycles/(300×10^6) cycles/sec = 4×10^{-6} sec = .004 ms

Step 3 : Find % of Processor Time

% Processor Time = (.004/1) × 100 = 0.4%

Term	Normal Usage	Usage as Power of 2
K (Kilo)	10^3	2^{10} = 1,024
M (Mega)	10^6	2^{20} = 1,048,576
G (Giga)	10^9	2^{30} = 1,073,741,824
T (Tera)	10^{12}	2^{40} = 1,099,511,627,776

References

- M. M. Mano, Computer System Architecture, Prentice-Hall